

What Belongs in an Agent's Context Window

Import Structure, Issue Text, and Two Controls That Change the Answer

NEIL LUNAVAT, Pimpri Chinchwad College of Engineering, India

An agent asked to change one file in an unfamiliar repository cannot read the repository. In our corpus the median repository is 286,385 tokens and 60.5% of tasks sit in one larger than a 200,000-token window; the files that actually matter are a median 2.98% of it. Something must choose. Two signals are available for the choice: the repository's import structure, and the words of the issue report. Graph-based context tools order by the first.

We compare both, and their combination, on 2,412 change tasks from 1,162 real pull requests in 112 Python repositories, scoring each ordering by how much of an evidence-based answer key it covers per line read. The comparison only has an answer once one says what "needed" means, so we grade against two keys and never merge them: the files *co-edited* in the same pull request, and the files that *define symbols the change uses*. Ranks below are over all twenty-three orderings we score, the oracle included.

One result survives every control we apply. Reciprocal-rank fusion of a cost-scaled personalised walk with issue-text-to-path matching is the best non-oracle ordering under *both* keys on the 702 pull requests that carry both (AUC 0.487 and 0.470). It beats the best pure lexical ordering by 0.055 and 0.067, and the best pure structural one by 0.097 and 0.039, all Holm-corrected with the repository as the unit.

The two single-signal effects that a less careful reading of our own tables would report do not survive. Graded on the full corpus, issue-to-path matching beats every purely structural ordering on the co-edited key (0.434 against 0.389 for the best walk) — but 40.2% of tasks have an issue that names a key file *as code*, and on the rest path matching falls from fifth to eleventh of the twenty orderings we test against one another, while *hops_lines*, which reads no issue text at all, takes its place in fifth. Graded on the full corpus, a walk following imports outward is second on the symbol key (0.491) against path matching's fifteenth (0.382) — but the symbol key also covers single-file pull requests, which have no co-edited key at all, and on a population matched to carry both keys the walk falls to eighth and is no longer distinguishable from path matching ($p_{\text{Holm}} = 1$). We report both controls at length because each dissolves an effect that we had ourselves previously written down as a finding.

What edge direction buys is also narrower than it first appears. Discarding direction costs nothing — an undirected walk and an outward one are indistinguishable on the symbol key ($p_{\text{Holm}} = 1$) — while reversing it is severe (+0.215 outward over inward, $p_{\text{Holm}} < 10^{-17}$). Both figures are over the full corpus, and both hold on the matched population. The finding is that walking *against* imports is bad, not that direction is the decisive structural choice. The symbol key is in addition partly circular with the methods it favours, being built by resolving names with *jedi*, which follows import edges; we state that at the outset rather than claim a symmetric discovery.

Two further results bound the problem. The two-hop neighbourhood of the seed, the usual justification for graph retrieval, holds 87.5% of the needed files but is also 56% of the repository and a median of 21,646 lines: it supplies recall, not precision, at a lift of only 1.57× over taking the same share of files at random. And the distance to a perfect ordering is large and measurable without running any model: an oracle puts the complete answer inside an 8,000-line budget for 97.5% of pull requests, whereas the best method we test reaches 58.5%. Since no agent can use a file it was never given, that gap is an upper bound on what better selection could add to any system under that budget — not an estimate of it, since an oracle reads only key files by construction. We argue that retrieval for code change must be reported per evidence source *and* per leakage stratum, that filename mentions in issue text are a first-class confound rather than a footnote, and that a cheap lexical signal belongs in any graph-based context tool that does not already have one. We make no claim about downstream repair rates; Section 6 pre-registers the experiment that would test one.

Author's address: Neil Lunavat, Pimpri Chinchwad College of Engineering, India, neillunavat3192@gmail.com.

2026. 1049-331X/2026/9-ART \$15.00

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

Additional Key Words and Phrases: code search, context retrieval, dependency graphs, software maintenance, empirical software engineering

ACM Reference Format:

Neil Lunavat. 2026. What Belongs in an Agent’s Context Window: Import Structure, Issue Text, and Two Controls That Change the Answer. *ACM Trans. Softw. Eng. Methodol.* 1, 1 (September 2026), 23 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

An agent is handed a bug report and a file to change. Before it can write anything it must decide what else to read, and it cannot simply read everything. In the corpus we study the median repository is 286,385 tokens; 74.8% of tasks sit in a repository larger than a 128,000-token window and 60.5% in one larger than 200,000. Meanwhile the files that carry the answer are a median 2.98% of the repository. The reduction available is roughly 34×, and every line spent on an irrelevant file is a line not spent on a relevant one. Something has to choose, and the choice is the subject of this paper.

A human faces a softer version of the same problem — attention rather than tokens — and we return to that reading in Section 5. The budget is what makes the question sharp, so we pose it in those terms throughout.

The dominant answer in tool building is the dependency graph. Aider builds a repository map from the call and definition graph; RepoGraph, LocAgent and CoSIL construct code graphs and walk them outward from a starting point. The premise is intuitive and rarely stated: what you need to read is near, in the graph, to what you are changing.

There is a second signal, and it is the one a human reaches for first. Asked to fix the login page, a developer thinks of the authentication files — not because `login.py` imports them, which it may well not, but because the words match. That signal sits in the issue report.

We are not the first to notice it. SWE-bench’s own sparse-retrieval baseline is BM25 with the issue as the query [9], Agentless [20] leans on file and directory names, and the agentic systems all read the issue text as part of their loop. What is missing is not the idea that words help. It is a measurement of how far they carry against structure, on a stated answer key, under a stated budget, and with the issue corpus’s habit of naming the answer controlled for. Every system in Table 1 reports end-to-end accuracy against a single key; none reports that comparison, and none reports how often its issues name the files it is scored on.

This paper compares the two. The comparison sounds simple and is not, because it has no answer until one says what it means for a file to have been “needed”. We use two evidence sources that can be recovered from the repository without human labelling: the files *co-edited* in the same pull request, and the files that *define symbols* the changed code references. Both are defensible. Neither is obviously the right one. They give opposite answers to the question this paper asks.

Research questions

- RQ1** How much of a repository is the seed’s two-hop neighbourhood, and how much recall does it buy over a same-sized random sample?
- RQ2** Do structural and lexical reading orders rank differently under the two evidence sources, and by how much?
- RQ3** How much does conditioning on the seed file add over a query-independent repository prior, and does the answer depend on the evidence source?
- RQ4** Do the two signals retrieve the same files, or different ones?
- RQ5** Does fusing them produce an ordering that is good under both evidence sources?

RQ6 What fraction of tasks could be solved at all under a given budget — that is, how much of the gap between current methods and a perfect ordering is headroom rather than a property of the tasks?

Contributions

- (1) A head-to-head comparison of import-graph proximity and issue-text retrieval as *reading orders* under a line budget, on 2,412 tasks from 1,162 real pull requests in 112 Python repositories, with both signals scored against the same two evidence sources.
- (2) Evidence that the conclusion of such a comparison is decided as much by the evidence source and by how the corpus is filtered as by the methods. Across twenty contending orderings the two keys induce different rankings, but we show that two of the largest apparent effects are properties of the comparison rather than of the signals: one is file-name leakage from the issue text, the other a population mismatch between the two keys. All claims are Holm-corrected with the *repository* as the unit, since pull requests cluster within repositories and treating them as independent inflates significance by many orders of magnitude.
- (3) A quantification of what graph proximity actually contributes: recall over half the repository, at a lift of 1.57×, with 4.2% precision at two hops.
- (4) A robustness criterion — worst-case rank across evidence sources — under which three of our five fusions beat every pure signal. The dividing line is not whether path matching is included, since four of the five include it and one of those still fails. It is whether the structural input is cost-scaled: the three that fuse from hops_lines or ppr_und_pl succeed, and the two built on the unscaled ppr_und do not.
- (5) A cost accounting in tokens rather than files: a corpus-wide calibration of Python source (8.50 tokens per line in aggregate, median 7.88), and a method-independent ceiling — the share of pull requests whose complete answer fits a given budget at all. An oracle ordering reaches 97.5% of pull requests within 8,000 lines where the best method we test reaches 58.5%, so the shortfall is headroom and not task difficulty.
- (6) A precise statement of what edge direction buys. Reversing a walk is severely harmful — following imports outward beats the same walk run inward by 0.215 AUC on the symbol key, or 0.123 on the population matched to both keys ($p_{\text{Holm}} < 10^{-14}$ in both cases, repository as the unit) — but *discarding* direction costs nothing, because the undirected walk common in this literature is indistinguishable from the outward one ($p_{\text{Holm}} = 1$ on the full corpus, 0.42 on the matched one). On the co-edited key direction is not merely unhelpful but harmful: all four directed variants rank seventeenth to twentieth, below a same-directory baseline that uses no graph at all, while all four undirected walks rank tenth to fourteenth, above it.
- (7) Two controls that each dissolve an effect we had previously reported as a finding, and which we recommend as standard practice. A *leakage* control that classifies whether an issue names a key file as code rather than merely sharing a word with it: 40.2% of tasks do, and on the rest issue-to-path matching falls from fifth to eleventh of the twenty contenders on the co-edited key. A *population* control that restricts both keys to the 702 pull requests carrying both, since a co-edited key requires a multi-file pull request while a symbol key does not: on the matched population the best directed walk falls from second to eighth on the symbol key and is no longer distinguishable from path matching.
- (8) A replication package with the graphs, tasks, issue-token bags, token counts, orderings, generated tables and analysis scripts.

2 BACKGROUND AND RELATED WORK

Dependency structure as a guide to change. Using program structure to decide what a change touches is old. Impact analysis [3] propagates from a changed element along dependencies; Robillard [18] ranks program elements by the topology of their dependencies; feature-location surveys [6] catalogue structural, textual and dynamic techniques and note that they are usually evaluated in isolation. Ko et al. [10] observed developers navigating largely by identifier names and search rather than by following dependencies.

Change history. Zimmermann et al. [26] and Ying et al. [23] mine co-change from version histories to suggest files that change together. Our co-edited evidence source is the same notion of relatedness, used as ground truth rather than as a method.

Graph-based context for agents. SWE-bench [9] and its successors [2, 24] made repository-scale change tasks a standard benchmark, and a family of systems now assembles context by walking a code graph: Aider’s repository map [1], RepoGraph [14], LocAgent [5] and CoSIL [8]. Agentless [20] is a notable counterexample that leans on file and directory names.

What these systems share is the ground truth they are judged against. Table 1 records it for every system whose evaluation section we read in the published PDF; each entry is quoted or paraphrased from that section, not from an abstract or a secondary description. Where localization accuracy is reported at all, the target is the set of files or functions touched by the gold patch — the SWE-bench convention, and exactly our co-edited source. That is the evidence source on which, in our results, structural proximity is the weaker of the two signals.

Concurrent work is questioning the key. The patch convention is under active revision, by at least four efforts that appeared while this study was running, and we want to be exact about how close they come. SWE-Explore [25] derives line-level ground truth “from independent agent trajectories that successfully solved the same issue, distilling the specific code regions their solution paths actually consulted”, and scores a ranked list “under a fixed line budget” — both the key and the cost model are close relatives of ours. ContextBench [11] annotates gold context by hand, 522,115 lines over 1,136 tasks, and measures recall, precision and efficiency rather than end-to-end resolution. AgentRetrievalBench [15] defines a file as relevant “when reading it would materially help the agent take the next correct step”, evaluated under a top- k or token budget; one of its four task types, ripple files beyond an anchored edit, is our co-edited source by another name. And GraphLocator [12] keeps the patch key but dual-annotates it, on the stated grounds that patch-derived ground truth misses locations.

We take this as corroboration of the premise rather than as a threat to it: several groups independently concluded in the same year that the patch key is inadequate. What one of them reports is the consequence we measure. Each proposes a better key and evaluates methods against it; we hold two keys already recoverable from existing benchmark data, and show that the choice between them *reverses the ranking of the methods themselves*, with every individual reversal significant under Holm correction. The contribution is not the observation that keys differ. It is the measurement of what the field’s default key costs the signal the field has invested in.

Retrieval. BM25 [17] remains the reference lexical ranking function; we use it unchanged, varying only the query.

3 STUDY DESIGN

The corpus, the graph builder, the task definition and the coverage-cost metric were fixed and tagged in version control before any method was run; the present study adds method families and

Table 1. What repository-context systems are graded against, read from each paper’s own evaluation section (2026-09-23). The upper block is the patch-based convention; the lower block is the concurrent work that departs from it.

System	Venue	Ground truth used for evaluation
<i>Files touched by the gold patch</i>		
LocAgent [5]	ACL 2025	“The affected files or functions ...as identified in the patches, are considered the target locations”
CoSIL [8]	ASE 2025	File- and function-level accuracy on SWE-bench Lite and Verified, i.e. the patched locations
Agentless [20]	FSE 2025	Correct if the edit covers a superset of the ground-truth patch locations, at line, function and file level
LingmaAgent [13]	FSE 2025	Fault locations extracted from the developer patch, at file and function level
SWE-Fixer [21]	ACL 2025	Precision and recall against the defective files of the resolving pull request; BM25 top-30 then a learned reranker
KGCompass [22]	2025	Files and code elements modified by the ground-truth patch, scored by Jaccard overlap
SWE-Adept [7]	2026	Correct only if <i>all</i> locations modified by the ground-truth patch appear in the top- <i>k</i>
SWERank [16]	2025	Every function modified in the pull request is a positive
CodeScout [19]	2026	Reuses LocAgent’s patch-processing scripts: modified files, modules and functions
RepoGraph [14]	ICLR 2025	“The percentage of problems where the edit locations match the ground truth patch”, at file, function and line level
<i>Something other than the gold patch alone</i>		
GraphLocator [12]	2025	Locations extracted from the fix patch, then dual-annotated and cross-reviewed because patch-derived ground truth “contains missing locations”
SWE-Explore [25]	2026	File and line intervals that successful agents actually <i>read</i> , intersected across several verified trajectories
ContextBench [11]	2026	Human-annotated context traced outward from the patch regions, pruned by a second annotator, validated by an LLM patching from it alone
AgentRetrievalBench [15]	2026	A file is relevant if reading it helps the agent’s next step; one of four task types is ripple files beyond an anchor
CodeR [4]	2024	End-to-end on SWE-bench, but its separate localization study uses golden locations marked by the authors

analyses on top of that frozen base. Changes made after that tag, and claims we have retracted, are recorded in the replication package rather than silently absorbed.

The freeze covers the corpus, the graphs, the task definition and the metric. It does not cover the method families: the fusion orderings, the directed walks and the lexical methods queried by issue text were all added afterwards, in response to what the first results showed. Fusion is the paper’s main recommendation and it is an *exploratory* result, not a pre-registered one. We say so here rather than let the word “pre-registered” cover more than it earns.

3.1 Repositories and tasks

We draw pull requests from SWE-bench, SWE-bench-Live, SWE-Gym, SWE-bench Pro, Loc-Bench and SWE-rebench, keeping Python repositories with a permissive licence. A pull request that edits *k* existing non-test source files that are nodes of the import graph yields *k* tasks: each edited file in turn becomes the *seed*, and the task is to order the remaining files of the repository. This gives 3,123 tasks from 1,873 pull requests in 113 repositories. Of these, 2,412 tasks — 1,162 pull requests in 112 repositories — have at least one key file inside the graph once the seed is removed, and

they are the ones scored here; the remaining 711 tasks have an empty key under both sources and are reported but not scored. Those 711 tasks come from 711 distinct pull requests, and 706 of them change a single file: with no second edited file there is no co-edited key, and the symbol key is empty whenever the changed code resolves to nothing outside itself — standard library, third-party packages, or names `jedi` could not resolve. That is 61.2% of all single-file tasks, which is a real limitation of the symbol key and not a filtering choice of ours. Within the scored set, 1,958 tasks (708 pull requests) have a co-edited key and 2,341 (1,156 pull requests) a symbol key.

The median repository contributes 131 graph nodes and the median key is 3 files totalling 1,515 lines, so the task is to find a handful of files among roughly a hundred.

3.2 Import graphs

Graphs are built per base commit with a custom ast walker over the repository at that commit. Nodes are source files; an edge $a \rightarrow b$ means a imports a module resolving to b . Package `__init__` files are not inserted as intermediaries, from `X import Y` resolves to the module, edges are unweighted, imports inside `TYPE_CHECKING` blocks and inside function bodies are kept, and test files are excluded. Per-file line counts come from the same pass and are the cost unit throughout.

3.3 Two evidence sources

co_edited The other non-test source files changed by the same pull request. This is what a developer in fact touched, and it is independent of the import graph.

symbol Files that define names the changed code references, resolved with a static analyser. This is what the change depends on.

Symbol resolution uses `jedi's goto with follow_imports=True`, which reaches a name's defining file by traversing import statements. This is worth stating plainly because it bounds what the symbol key can show: the key is constructed by walking the import graph, so a method that walks the import graph is being scored against a target built from the same substrate. We report it because it is the natural static notion of "what this code depends on", not because it is a neutral arbiter between the two method families.

The seed file is removed from both. We deliberately do not merge them: the whole question is whether the choice matters. It does, and the union is not a neutral compromise — the symbol key contributes 7,675 file references against 4,458 for co-editing, so a union largely inherits the structural source's verdict.

A third source — files actually executed by the tests that the fix repairs — is defined in our protocol and was collected for 113 pull requests before we stopped. We exclude it rather than report it: a source covering under a tenth of the corpus cannot be compared against two that cover all of it, and reporting it alongside them would invite exactly the conflation this paper argues against. The partial collection is in the replication package so that it is visible rather than quietly dropped, and completing it is the obvious extension.

3.4 Reading orders

Every method receives the graph at the base commit, the seed file, per-file identifier bags and the issue text, and returns an ordering of all other nodes. Table 2 lists them by family. Identifier bags are built by splitting every identifier in the file on underscores and camel-case boundaries and lower-casing; the issue text is tokenised identically, so text is matched against code over one vocabulary. BM25 is used with the standard $k_1 = 1.5$, $b = 0.75$ and differs between methods only in the query. Fusion is reciprocal-rank fusion with $k = 60$.

Table 2. Reading orders, by family. Structural methods use the seed and the graph; global methods use neither the seed nor the issue; lexical methods use no graph.

Family	Method	Ordering
structure	bfs_und	breadth-first from the seed, undirected
	hops_lines	by (hop distance from seed, file length)
	ppr_und	personalised PageRank from the seed, undirected
	ppr_und_pl	the same, divided by file length
	ppr_out	the same walk along import edges: what the seed depends on
	ppr_out_pl	the same, divided by file length
	ppr_in	the same walk against import edges: what depends on the seed
	ppr_in_pl	the same, divided by file length
global	pagerank	PageRank of the whole graph; ignores seed and issue
	indegree	number of files importing this one
lexical	bm25_seed	BM25, query = identifiers of the seed file
	path_seed	BM25 over paths, query = the seed's path tokens
	bm25_issue	BM25, query = identifiers of the issue text
	bm25_issue_pl	the same, divided by file length
	path_issue	BM25 over file paths, query = issue tokens
fusion	rrf_hops_path	RRF of hops_lines and path_issue
	rrf_hops_issue_path	RRF of hops_lines, bm25_issue, path_issue
	rrf_pprpl_issue_path	RRF of ppr_und_pl, bm25_issue, path_issue
	rrf_ppr_issue	RRF of ppr_und and bm25_issue
	rrf_ppr_issue_path	RRF of ppr_und, bm25_issue, path_issue
reference	same_dir	files in the seed's directory first
	random	uniformly shuffled: the floor
	oracle	key files only, cheapest first: the ceiling

Two points of fairness are worth stating. The structural methods are *given* a correct answer for free — the seed is one of the files the pull request changed. The issue-text methods are given no seed at all. And the metric charges by lines read, so we include cost-scaled variants (suffix _pl, score divided by file length) to separate “ranks the right files highly” from “happens to prefer short files”.

3.5 Metric

An ordering is read from the top until a line budget is exhausted; coverage is the fraction of the key that has been read. We evaluate at budgets {250, 500, 1000, 2000, 4000, 8000, 16000, 32000} and report the AUC, the mean coverage over those eight doublings, so that a method must be good at both tight and generous budgets. We also report the more directly interpretable quantity: the number of lines that must be read before half the key has been covered.

Orderings with ties are re-drawn three times per task with distinct seeds and the coverage curves averaged.

The budget is denominated in lines because the pre-registered design fixed it that way at freeze-1, and we do not change a frozen metric after seeing results. Lines are nevertheless the wrong unit for a context window, so we measure the conversion rather than assume it: across the 121,075 distinct

file versions in the corpus, tokenised with the `cl100k_base` byte-pair vocabulary, Python source runs to 8.50 tokens per line in aggregate and a median of 7.88 per file, with a 5th–95th percentile range of 5.70–10.90. Absolute counts shift by a small constant factor between tokenizer families; the ratios we report do not depend on the choice. A budget of 8,000 lines is therefore roughly 68,000 tokens, and the eight budgets span roughly 2,100 to 272,000 tokens — from a fraction of a small context window to more than a large one.

3.6 Statistics

Task AUCs are averaged within a pull request first, so a change touching many files does not count many times. Paired comparisons use the Wilcoxon signed-rank test with the Vargha–Delaney A_{12} effect size, Holm-corrected within each family of comparisons. Confidence intervals are 2,000-resample bootstraps resampling *repositories*, not tasks, so that a few large repositories cannot manufacture significance.

The paired tests are reported with the **repository** as the unit: the pull-request scores are averaged within a repository and the test runs over the 112 repositories. Pull requests are not independent draws — they cluster in repositories that differ in size, layout convention and issue-writing culture — and a test that ignores the clustering counts the same evidence many times. The difference is not cosmetic. The same comparison that yields $p < 10^{-100}$ over 1,156 pull requests yields $p < 10^{-20}$ over 112 repositories. The effect sizes are essentially unchanged, which is the point: the conclusions survive, and only the arithmetic of the p -values was ever overstated. We report the pull-request-level tests alongside, in the replication package, so the two can be compared.

Because a co-edited key requires a pull request that touches more than one graph file while a symbol key does not, the two keys do not cover the same pull requests. Wherever we compare across keys we therefore report the comparison twice: over each key’s full population, and over the 702 pull requests that carry both (Section 4.3).

4 RESULTS

4.1 RQ1: the two-hop neighbourhood is half the repository

“The files you need are within two hops” is true here: 86.7% of symbol-key files and 88.1% of co-edited files lie within two undirected hops of the seed (Table 3). Taken alone this reads as strong evidence for graph retrieval.

The other half of the statistic undoes it. In the median task the two-hop ball holds 68 of the repository’s 131 files and 21,646 lines — 56% of the files on average, and more than five times a 4,000-line budget. Its precision is 4.2%. Against a random sample of the same size it lifts recall by a factor of 1.57.

The two-hop neighbourhood is therefore not a filter. It is a statement that the files you need are somewhere in about half the repository — which is exactly the regime in which the *ordering* decides everything and the graph has already said all it has to say.

4.2 RQ2: the evidence source decides the winner

Table 5 is the central result. Under the co-edited key the top five orderings are four fusions and one lexical method; the best purely structural ordering is tenth, and the query-independent priors are twenty-first and twenty-second of twenty-three, below `same_dir` and barely above random. Under the symbol key the table inverts: the top four are three structural orderings and one fusion, and the best lexical ordering is fifteenth.

Table 3. Where the key files sit relative to the seed, and how big the two-hop ball is. Hop shares are pooled proportions over all key-file references; the ball columns are medians over 2,412 tasks.

Key files at hop distance	1	2	3+	unreachable
co_edited	39.0%	49.1%	9.1%	2.8%
symbol	45.3%	41.4%	9.1%	4.2%
<i>The two-hop ball:</i> 68 files, 21,646 lines, 56% of the repository				
recall 87.5%, precision 4.2%, recall lift over a same-sized random sample 1.57×				
<i>The key:</i> 3 files, 1,515 lines				

The individual reversals are large and significant after Holm correction with the repository as the unit. Issue-text path matching beats cost-scaled personalised PageRank by 0.059 AUC on co-edited files ($A_{12} = 0.403$ for the walk, $p_{\text{Holm}} < 10^{-4}$) and loses to it by 0.100 on symbol files ($A_{12} = 0.671$, $p_{\text{Holm}} < 10^{-8}$).

We rest the reversal on those paired tests rather than on a rank correlation. The correlation is low – Spearman $\rho = 0.24$ over the twenty contenders ($p = 0.31$), and $\rho = 0.37$ over all twenty-two non-oracle orderings ($p = 0.09$) – but with twenty points a non-significant ρ is weak evidence of anything, and its value moves with which methods are included. The Holm-corrected pairwise reversals do not have that weakness.

Two things about Table 5 are *not* safe to read off it, and the next two subsections are about them. The two columns are computed over different pull requests, and the left column is computed over a corpus in which many issues name the answer outright. Each of those turns out to account for one of the two halves of the reversal.

Figure 1 shows the same reversal as coverage curves. Table 6 states it in the unit a context budget is actually spent in: to reach half the co-edited files a reader following cost-scaled personalised PageRank consumes a median of 6,155 lines, against 4,838 following issue-text path matching and 3,342 following the best fusion. On symbol files the same three numbers are 3,483, 6,638 and 3,957 – the walk and the words trade places, and the fusion is close to the better of the two in both.

4.3 The reversal is smaller on a matched population

A co-edited key exists only for a pull request that changes more than one file in the graph. A symbol key does not need that, and 444 of the 1,156 pull requests with a symbol key change a single file. The two columns of Table 5 are therefore scored on different samples, and single-file changes are exactly the case where a seed's import neighbourhood is most of what there is to find. The comparison has to be repeated on a population that carries both keys.

Table 4 does that, on the 702 pull requests that have both. On the co-edited key nothing moves: no method changes rank by more than one place. On the symbol key the structural family loses most of its advantage. Cost-scaled personalised PageRank along import edges falls from second to eighth (0.491 to 0.417), the undirected form from third to fifth, and hops_lines from fourth to sixth, while issue-to-path matching rises from fifteenth to ninth. The three top places under both keys are then held by the same three fusions.

The statistical claim moves with the ranks. On the full corpus a walk along import edges beats path matching on the symbol key by 0.101 ($p_{\text{Holm}} < 10^{-8}$, repository as the unit). On the matched population the same comparison gives +0.008 and $p_{\text{Holm}} = 1$; for hops_lines against path matching it gives +0.019 and $p_{\text{Holm}} = 1$. We therefore do not claim that structural proximity beats issue-text retrieval on the symbol key. On a like-for-like sample the two are indistinguishable, and what remains of the reversal is the co-edited half – which the next control also qualifies.

Table 4. The two keys re-scored on the 702 pull requests that carry both, top ten under each. Δ is the change in rank against the full population of Table 5; a positive number means the method does worse once the samples are matched.

Method	Fam.	AUC	#	Δ
oracle	ref	.774	1	–
rrf_pprpl_issue_path	fus	.487	2	–
rrf_hops_issue_path	fus	.480	3	–
rrf_hops_path	fus	.457	4	–
rrf_ppr_issue_path	fus	.445	5	–
path_issue	lex	.435	6	–
bm25_issue	lex	.409	7	–
rrf_ppr_issue	fus	.394	8	–1
bm25_seed	lex	.393	9	+1
hops_lines	str	.389	10	–
oracle	ref	.779	1	–
rrf_pprpl_issue_path	fus	.470	2	–4
rrf_hops_path	fus	.456	3	–2
rrf_hops_issue_path	fus	.454	4	–4
ppr_und_pl	str	.436	5	+2
hops_lines	str	.425	6	+2
rrf_ppr_issue_path	fus	.421	7	–4
ppr_out_pl	str	.417	8	+6
path_issue	lex	.404	9	–6
ppr_und	str	.395	10	+1

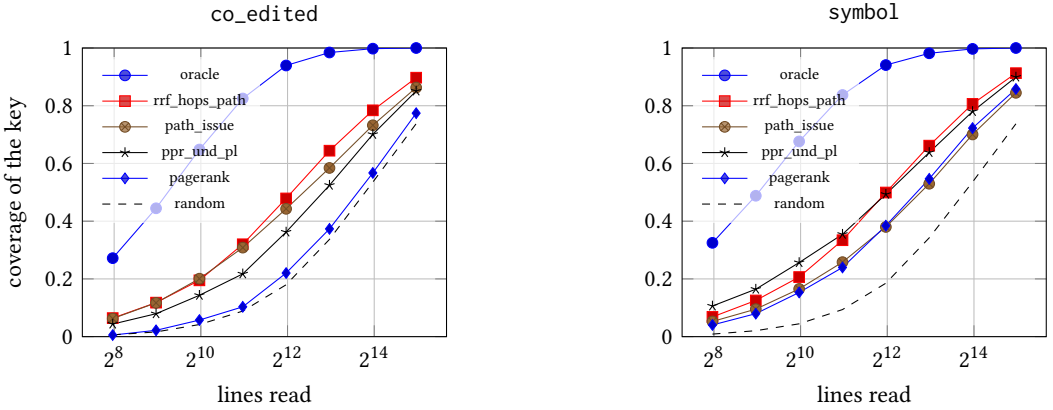


Fig. 1. Mean coverage against lines read. The same five orderings change places between the two evidence sources: issue-text path matching tracks the best fusion on co-edited files and falls below the query-independent prior on symbol files, while cost-scaled personalised PageRank does the reverse.

4.4 Direction: reversing a walk is costly, discarding it is free

The walks above are undirected, as they are in most of this literature. An import edge has a direction, so we add both orientations of the personalised walk, cost-scaled and not.

Table 5. Mean AUC by evidence source, pull request as the unit, with bootstrap 95% confidence intervals over repositories. The same twenty-three orderings, ranked twice.

co_edited (708 PRs)				symbol (1,156 PRs)			
#	Method	Fam.	AUC [CI]	#	Method	Fam.	AUC [CI]
1	oracle	ref	0.773 [.750,.796]	1	oracle	ref	0.793 [.772,.813]
2	rrf_pprpl_issue_path	fus	0.486 [.455,.517]	2	ppr_out_pl	str	0.491 [.463,.517]
3	rrf_hops_issue_path	fus	0.480 [.448,.510]	3	ppr_und_pl	str	0.489 [.460,.517]
4	rrf_hops_path	fus	0.457 [.425,.489]	4	hops_lines	str	0.477 [.447,.505]
5	rrf_ppr_issue_path	fus	0.445 [.414,.475]	5	rrf_hops_path	fus	0.476 [.449,.502]
6	path_issue	lex	0.434 [.405,.465]	6	rrf_pprpl_issue_path	fus	0.474 [.445,.501]
7	bm25_issue	lex	0.407 [.379,.435]	7	ppr_out	str	0.458 [.430,.487]
8	bm25_seed	lex	0.393 [.362,.425]	8	rrf_hops_issue_path	fus	0.457 [.429,.483]
9	rrf_ppr_issue	fus	0.393 [.363,.425]	9	ppr_und	str	0.446 [.417,.475]
10	hops_lines	str	0.389 [.354,.423]	10	bfs_und	str	0.420 [.391,.447]
11	ppr_und_pl	str	0.371 [.338,.403]	11	rrf_ppr_issue_path	fus	0.420 [.392,.448]
12	path_seed	lex	0.371 [.338,.404]	12	pagerank	glo	0.410 [.381,.439]
13	bfs_und	str	0.362 [.330,.395]	13	indegree	glo	0.398 [.371,.425]
14	ppr_und	str	0.357 [.325,.391]	14	rrf_ppr_issue	fus	0.390 [.361,.421]
15	bm25_issue_pl	lex	0.354 [.318,.389]	15	path_issue	lex	0.382 [.352,.412]
16	same_dir	ref	0.329 [.298,.360]	16	path_seed	lex	0.365 [.336,.392]
17	ppr_out_pl	str	0.327 [.296,.358]	17	bm25_issue_pl	lex	0.355 [.324,.386]
18	ppr_out	str	0.312 [.282,.344]	18	bm25_issue	lex	0.340 [.311,.368]
19	ppr_in_pl	str	0.310 [.279,.341]	19	bm25_seed	lex	0.336 [.305,.365]
20	ppr_in	str	0.308 [.278,.339]	20	same_dir	ref	0.320 [.290,.349]
21	pagerank	glo	0.271 [.240,.301]	21	ppr_in_pl	str	0.277 [.248,.303]
22	indegree	glo	0.271 [.240,.301]	22	ppr_in	str	0.269 [.242,.295]
23	random	ref	0.248 [.220,.277]	23	random	ref	0.248 [.220,.276]

Table 6. Median lines that must be read before half, then all, of the key is covered. Lower is better.

Method	to half the key		to all of it	
	co_edited	symbol	co_edited	symbol
oracle	524	440	1,150	1,349
rrf_pprpl_issue_path	3,342	3,957	7,213	9,563
rrf_hops_path	3,962	3,715	8,073	9,636
ppr_out_pl	8,655	3,924	15,396	12,386
ppr_und_pl	6,155	3,483	11,644	11,303
hops_lines	5,620	4,080	11,454	11,582
path_issue	4,838	6,638	11,072	14,504
bm25_issue	5,012	7,451	10,926	16,699
random	14,557	14,713	21,026	22,643

The result is asymmetric, and the asymmetry is the finding. Running the walk *along* import edges — towards what the seed depends on — places ppr_out_pl second of twenty-three on the symbol key at 0.491. Running it *against* them collapses to ppr_in_pl at 0.277, twenty-first. The like-for-like gap is 0.123 on the matched population and 0.215 on the full one, in both cases the largest separation between two structural orderings anywhere in our results ($p_{\text{Holm}} < 10^{-14}$, repository as the unit).

But the undirected walk loses almost nothing. On the symbol key `ppr_und_pl` scores 0.489 against `ppr_out_pl`'s 0.491, a difference of 0.002 that is not significant on the full corpus ($p_{\text{Holm}} = 1$) and reverses in sign, slightly, on the matched one. So the practice this literature has settled on — throw the direction away — is not the mistake it might appear to be from the spread of the table. Walking backwards is the mistake.

On the co-edited key direction is worse than useless. All four directed variants rank seventeenth to twentieth, below `same_dir`, a baseline that orders files by directory and consults no graph at all; all four undirected walks rank tenth to fourteenth, above it. Committing to an orientation costs 0.044 to 0.061 AUC there ($p_{\text{Holm}} < 10^{-3}$).

The explanation is the same one that qualifies the symbol key throughout. That key is built by resolving names with `jedi`, which follows imports in exactly one direction, so a walk in that direction is measuring something close to the key's own construction. The co-edited key records a human decision about what belonged in one change, and no orientation of the import graph recovers it well.

4.5 RQ3: what the seed is worth depends on the source too

A query-independent PageRank prior — a ranking that never looks at the seed or the issue, and simply says which files matter in this repository — reaches 0.410 on symbol files against 0.489 for the best cost-scaled personalised walk, which does look at the seed. That is 84% of the personalised score, and far above random (0.248; $A_{12} = 0.758$, $p_{\text{Holm}} < 10^{-16}$). On that evidence source, much of what the personalised walk delivers is a repository prior.

Under the co-edited key the same prior scores 0.271 against a random floor of 0.248. The difference is significant but negligible ($A_{12} = 0.552$, $p_{\text{Holm}} = 0.001$), and the personalised walk beats it by 0.101 ($A_{12} = 0.666$). Knowing which file is being changed therefore matters a great deal for predicting what else was changed, and rather little for predicting what the code calls, where knowing the repository's hubs is nearly as good.

This is worth stating plainly, because the opposite claim is easy to make from a single source: whether “the graph knows what you are working on” appears true or false is itself a function of the answer key.

4.6 RQ4: the two signals name different files

The reversal is not two methods disagreeing about the order of one list. At $k = 10$ the top-10 sets of personalised PageRank and issue-text BM25 have a mean Jaccard overlap of 0.187, and of personalised PageRank and issue-text path matching only 0.102; at $k = 50$ these rise to 0.508 and 0.412. For comparison, personalised PageRank and the query-independent prior overlap at 0.364 at $k = 10$: a seed-conditioned walk resembles the repository prior more than it resembles a lexical ranking.

Table 7 follows individual key files. At $k = 20$, 24.4% of co-edited files are found only by the lexical family and 9.6% only by the structural family; on symbol files the asymmetry flips (19.9% structure-only against 16.5% words-only). Under either source about a sixth of key files are found by neither family at $k = 20$, which is where the remaining headroom to the oracle lives.

4.7 RQ5: fusion is the only ordering that survives both sources

If the evidence source is not known in advance — and for a tool shipped to users it is not — the quantity to optimise is the worst rank a method attains across sources. Table 8 ranks by it. Three of the five fusions beat every pure signal. The two that do not — `rrf_ppr_issue` and `rrf_ppr_issue_path` — are not distinguished by whether they include path matching, since four of the five do and one of those still fails. They are the two built on the unscaled `ppr_und`. The three

Table 7. How each key file is reached within the top k of the structural family (ppr_und, hops_lines) and the lexical family (bm25_issue, path_issue).

Source	k	both	structure only	words only	neither
co_edited	10	32.2%	11.4%	30.0%	26.4%
co_edited	20	50.7%	9.6%	24.4%	15.3%
co_edited	50	76.7%	5.9%	13.4%	4.0%
symbol	10	28.5%	23.3%	20.4%	27.8%
symbol	20	47.4%	19.9%	16.5%	16.2%
symbol	50	72.9%	11.5%	9.8%	5.9%

Table 8. Worst rank attained across the two evidence sources. Ranks are those of Table 5, over all twenty-three orderings with the oracle included, so the best attainable worst rank here is 2. Lower is better. The three fusions built on a cost-scaled structural input beat every pure signal; the two built on the unscaled walk do not.

Method	Fam.	co_edited	symbol	worst
rfr_hops_path	fus	4	5	5
rfr_pprpl_issue_path	fus	2	6	6
rfr_hops_issue_path	fus	3	8	8
hops_lines	str	10	4	10
ppr_und_pl	str	11	3	11
rfr_ppr_issue_path	fus	5	11	11
bfs_und	str	13	10	13
ppr_und	str	14	9	14
rfr_ppr_issue	fus	9	14	14
path_issue	lex	6	15	15
path_seed	lex	12	16	16
bm25_issue_pl	lex	15	17	17
ppr_out_pl	str	17	2	17
bm25_issue	lex	7	18	18
ppr_out	str	18	7	18
bm25_seed	lex	8	19	19
same_dir	ref	16	20	20
pagerank	glo	21	12	21
ppr_in_pl	str	19	21	21
indegree	glo	22	13	22
ppr_in	str	20	22	22
random	ref	23	23	23

that survive all fuse from a cost-scaled structural input, either hops_lines or ppr_und_pl. Dividing by file length before fusing is what makes the difference. The best, rfr_hops_path, is nothing more than reciprocal-rank fusion of hop-distance ordering with issue-tokens-against-file-paths, and is fourth under one source and fifth under the other.

The insurance is close to free. Against pure structure (hops_lines) it gains 0.067 AUC on co-edited files ($A_{12} = 0.602$, $p_{\text{Holm}} < 10^{-11}$) and gains 0.003 on symbol files ($p_{\text{Holm}} = 1$); against ppr_und_pl it gains 0.087 ($p_{\text{Holm}} < 10^{-13}$) and loses 0.008 ($p_{\text{Holm}} = 1$). The lexical signal adds a large amount where structure is weak and costs nothing where structure is strong.

Table 9. Share of pull requests whose *entire* key fits within a token budget. This is a ceiling: no ordering can exceed it, and no agent can act on files outside it.

Token budget	co_edited (%)	symbol (%)
4,000	34.1	31.5
8,000	54.2	50.8
16,000	75.1	72.5
32,000	90.3	87.3
64,000	96.9	95.5
128,000	99.5	99.4
200,000	99.9	99.9
1,000,000	100.0	100.0

4.8 RQ6: what a budget rules out, and what it does not

A coverage number says how much of the key a method found. It does not say whether finding all of it was possible. If the needed files together exceed the budget, no ordering of them fits and no downstream system can succeed, however capable: a model cannot use a file it was never given. That bound is a property of the task and the budget alone, so it can be measured once and applied to every method at once — and, usefully here, without running a model.

Table 9 reports it. The complete answer fits a 32,000-token budget for 90.3% of pull requests under the co-edited key and 87.3% under the symbol key, and a 64,000-token budget for over 95% of both. The task is not, in general, impossible at realistic budgets.

We are careful about what this licenses. The oracle reads only key files, so that it fits a small key into a small budget is close to arithmetic: the ceiling shows the keys are small, not that the remaining distance is reachable from the signals available. What it does rule out is the competing explanation — that current methods score as they do because the answers do not fit — and that is worth ruling out, because it is the explanation a reader reaches for first. The gap between 58.5% and 97.5% is therefore an upper bound on what better selection could add, not an estimate of it.

Against that ceiling, current orderings fall a long way short. Table 10 counts the pull requests each method takes to *complete* coverage at 8,000 lines, a budget at which an oracle succeeds on 97.5% of them. The best method on the co-edited key, `rnf_hops_issue_path`, reaches 58.5%; the best on the symbol key, `hops_lines`, reaches 51.0%. Roughly forty points of the gap is therefore attributable to the ordering rather than to the budget or the task. The ceiling is partly definitional — an oracle that reads only key files must win — but its value is the converse reading: at 8,000 lines the budget is almost never the binding constraint, so a failure to cover the key is a failure of the ordering and not of the budget.

Two things are worth noting. The reversal survives the stricter criterion: `bm25_issue` loses 14.9 points and `path_issue` 11.5 points moving from the co-edited key to the symbol key, while the outward walks gain the most (`ppr_out` 15.2 points, `ppr_out_p1` 14.7) and `pagerank` gains 11.5. The gain is not a property of the structural family as such: the two inward walks lose 11.2 and 11.7 points, more than `path_issue` does. It is a property of walking the direction the key was built along. And the fusion orderings again sit at or near the top of both columns, which is the same conclusion RQ5 reached by AUC, reached here by a measure that ignores partial credit entirely.

Table 10. Share of pull requests taken to *complete* coverage within 8,000 lines. The oracle row is the ceiling of Table 9 expressed in the frozen unit. The reversal of Table 5 reappears under this stricter criterion.

Ordering	co_edited (%)	symbol (%)
oracle	97.5	95.9
rff_hops_issue_path	58.5	49.3
rff_pprpl_issue_path	56.5	49.1
rff_hops_path	54.5	50.3
rff_ppr_issue_path	53.4	45.0
bm25_issue	51.8	36.9
rff_ppr_issue	50.6	44.6
bm25_seed	48.8	36.6
path_issue	48.4	36.9
hops_lines	48.1	51.0
ppr_und_pl	45.1	50.0
ppr_und	45.0	50.1
bm25_issue_pl	39.8	33.9
path_seed	39.6	36.3
bfs_und	36.6	40.1
ppr_out_pl	35.4	50.1
ppr_out	34.5	49.7
ppr_in	34.0	22.8
ppr_in_pl	33.9	22.3
pagerank	31.9	43.4
indegree	28.2	38.9
same_dir	27.8	24.2
random	15.3	14.3

4.9 Sensitivity: are the lexical methods just copying paths out of the issue?

Issue reports often contain stack traces, and a stack trace names files. If the lexical advantage came from reports quoting the paths of the files that had to change, it would be an artefact of the benchmark rather than a property of language.

Largely it does, and the effect is big enough that we report it as a result rather than as a reassurance.

The full path of a key file appears verbatim in the issue text for 12.6% of tasks, and a bare file stem for 48.5%. Path BM25 tokenises a path into lowercased subtokens, so a stem is enough to match and the full-path control is far too weak. But a bare stem is also too strong a criterion on its own: an issue about “models” and a key file called `models.py` share a word because a project’s prose and its filenames draw on one domain vocabulary, and using that overlap is retrieval, not leakage. We therefore classify every stem occurrence by how it is written. A mention is *explicit* if it appears as code — `core.py`, `simbad/core`, a dotted module path, inside backticks, or inside a fenced or indented block — and *incidental* if the stem occurs only as an ordinary English word. Explicit mentions cover 40.2% of tasks; a further 11.7% have incidental matches only.

That gives three nested strata, reported in Table 11: issues quoting no full key path (2,108 tasks, 668 pull requests on the co-edited key), issues naming no key file as code (1,442 tasks, 549 pull requests), and the strictest set containing no key stem at all (1,243 tasks, 509 pull requests). We treat the middle one as primary. It removes the leakage channel while leaving the vocabulary signal in place, and it does not select as heavily for vague issues as the strictest does.

Table 11. AUC and rank (in brackets) under the three nested leakage controls: issues quoting no full key path, issues naming no key file as code, and the strictest set containing no key stem at all. Ranks are within the twenty contenders on that stratum.

Method	no full path		no explicit mention		no path or stem	
	co_edited	symbol	co_edited	symbol	co_edited	symbol
rrf_pprpl_issue_path	.466 (1)	.463 (5)	.431 (1)	.437 (7)	.412 (2)	.421 (7)
rrf_hops_path	.440 (3)	.468 (4)	.410 (3)	.444 (6)	.391 (3)	.429 (6)
ppr_out_pl	.325 (15)	.494 (1)	.336 (14)	.498 (1)	.331 (14)	.497 (1)
ppr_und_pl	.372 (10)	.493 (2)	.378 (9)	.495 (2)	.371 (9)	.495 (2)
hops_lines	.389 (8)	.479 (3)	.394 (5)	.484 (3)	.385 (5)	.483 (3)
path_issue	.407 (5)	.367 (12)	.361 (11)	.327 (14)	.336 (13)	.304 (17)
bm25_issue	.396 (6)	.332 (16)	.382 (7)	.321 (16)	.379 (8)	.320 (15)
pagerank	.274 (19)	.414 (9)	.280 (19)	.412 (9)	.283 (19)	.414 (8)

The result is the same under both real controls, which is what makes it credible. Ranks in this subsection are over the twenty contenders, the scale Table11 uses, on which `path_issue` is fifth on the full corpus. Under the weak full-path control it barely moves, falling from 0.434 to 0.407 and holding fifth. Under the primary control it falls to 0.361 and *eleventh*; under the strictest, to 0.336 and thirteenth. `hops_lines` — pure structure, no issue text — rises to fifth in both, at 0.394 and 0.385, ahead of every pure lexical ordering. Because the middle and the strictest strata agree, the finding is not an artefact of over-correcting: it does not depend on discarding incidental vocabulary overlap. On the symbol key nothing moves in any stratum: `ppr_out_pl`, `ppr_und_pl` and `hops_lines` are first, second and third in all three.

We draw two conclusions and decline a third. First, the lexical advantage on the co-edited key is substantially an effect of issue reports naming the files that change, in code, in two tasks out of five. Second, that is not purely an artefact: naming does happen in real reports, so a tool that exploits it exploits something real, and the honest statement is conditional rather than universal. What we do *not* conclude — and what an earlier draft of this paper did conclude — is that words beat structure on co-edited files in general. That claim holds on the full corpus and fails under both of the controls we regard as meaningful.

What survives is fusion. The three cost-scaled fusions hold the top three places on co-edited files in every stratum, with `rrf_pprpl_issue_path` first at 0.431 in the primary stratum and `rrf_hops_issue_path` first at 0.414 in the strictest. The comparable leakage figures for the seed itself are 10.3% full path and 35.1% stem.

The `path_seed` control separates “issue text” from “path matching” more directly: the same path-BM25 ranking, queried by the seed’s own path instead of the issue, scores 0.371 on co-edited files against 0.434 for the issue query ($+0.064$, $p_{\text{Holm}} < 10^{-14}$), and the two are indistinguishable on symbol files ($+0.018$, $p_{\text{Holm}} = 0.30$). The gain is attributable to the issue text, and only where the target is what developers changed.

4.10 Sensitivity: does the reversal survive in large repositories?

The corpus skews small — the median repository contributes 131 graph nodes — so the reversal has to be shown not to be an artefact of repositories one can almost read whole. Table 12 repeats the ranking within quartiles of repository size (quartile boundaries 73, 131 and 246 nodes; the largest repository has 2,487). The median of 131 graph nodes and the median of 286,385 tokens per repository are consistent: they imply roughly 2,200 tokens, or some 260 lines, per file, which is an ordinary Python module.

Table 12. Mean AUC (rank among the twenty contenders) by quartile of repository size in graph nodes. Q1 ≤ 73 , Q2 ≤ 131 , Q3 ≤ 246 , Q4 up to 2,487.

Method	co_edited				symbol			
	Q1	Q2	Q3	Q4	Q1	Q2	Q3	Q4
rrf_pprpl_issue_path	.574 (1)	.487 (1)	.418 (1)	.369 (1)	.568 (2)	.476 (1)	.403 (4)	.375 (2)
rrf_hops_path	.552 (3)	.465 (3)	.397 (3)	.330 (4)	.562 (5)	.473 (2)	.409 (3)	.362 (4)
ppr_out_pl	.473 (14)	.359 (15)	.268 (14)	.168 (15)	.562 (4)	.450 (6)	.414 (2)	.372 (3)
ppr_und_pl	.497 (12)	.399 (8)	.316 (10)	.241 (11)	.568 (3)	.467 (3)	.423 (1)	.387 (1)
hops_lines	.517 (6)	.412 (6)	.329 (8)	.244 (10)	.569 (1)	.454 (5)	.402 (5)	.343 (7)
path_issue	.537 (5)	.437 (4)	.372 (5)	.302 (5)	.515 (12)	.416 (9)	.324 (11)	.257 (12)
bm25_issue	.516 (7)	.402 (7)	.334 (6)	.287 (6)	.476 (15)	.367 (15)	.273 (15)	.233 (14)
pagerank	.442 (19)	.299 (19)	.199 (19)	.113 (19)	.521 (10)	.370 (14)	.333 (10)	.286 (11)

The pattern of Table eftab:main holds in every quartile, so neither it nor the qualifications of Sections efsec:shared and efsec:leak are artefacts of repository size. It does not widen monotonically with size: the co-edited-minus-symbol rank gap for ppr_und_pl runs 9, 5, 9, 10 across the quartiles and for hops_lines 5, 1, 3, 3, so the right claim is stability, not a trend. In the largest quartile, issue-text path matching is fifth on co-edited files and twelfth on symbol files, while cost-scaled personalised PageRank is eleventh and first respectively. The query-independent prior degrades fastest of all: on co-edited files in the largest quartile it scores 0.113 against a random floor of 0.089, which is to say that in the repositories where retrieval matters most, knowing only which files are central is worth almost nothing. rrf_hops_path never falls below fifth in any of the eight cells, and is third or better in five of them.

5 DISCUSSION

What we can and cannot say about the two keys. The tidy version of this paper would be that the import graph answers the compiler's question and the issue text answers the developer's, and that the field has been grading the first against the second. Our own controls will not support the tidy version, and it is worth being exact about what is left.

On the symbol key, structural methods win on the full corpus and are indistinguishable from issue-to-path matching once the population is matched to the co-edited key (Section 4.3). That key is also built by following imports, so we would have discounted a win there in any case. On the co-edited key, issue-to-path matching wins on the full corpus and loses to pure structure once tasks whose issues name a key file in code are removed (Section 4.9). Each half of the reversal, in other words, has an explanation that is not about the signals at all — one is a sampling artefact, the other a leakage artefact — and an earlier draft of this paper reported both halves as findings.

What remains is weaker but real. The two keys still induce different rankings, and a method tuned on one is not thereby good on the other. But the honest summary of the single-signal comparison is that neither family dominates once both confounds are controlled, and the interesting result is elsewhere.

Fusion is the result. It is the only claim in this paper that survives every control we applied. Reciprocal-rank fusion of a cost-scaled walk with issue-to-path matching is the best non-oracle ordering under both keys on the matched population, where it beats the best pure lexical ordering by 0.055 and 0.067 and the best pure structural one by 0.097 and 0.039 at the repository level, and it holds the top three places on co-edited files in all three leakage strata. It needs no model, no index beyond the file list, and no extra hop through the graph.

Two caveats keep this from being stronger than it is. The fusions are the only methods given both the seed and the issue, so part of their advantage is simply more input rather than genuine complementarity; the overlap analysis (Section 4.6) argues for complementarity, but a fusion of structure with a non-issue lexical signal would separate the two cleanly and we have not run one. And the fusion family and the worst-rank criterion were both added after the design freeze (Section 3), so they are exploratory, and a held-out split of repositories would be the right way to confirm them.

The field's default key, restated carefully. Every graph-based localization system we could check (Table 1) is graded against the files the gold patch touched — our co_edited source. On that source the best purely structural ordering ranks tenth of the twenty-three orderings we score, behind matching issue words against file paths, a method with no graph, no model and no seed. That gap narrows and then reverses once filename leakage is removed, which is itself the point: the comparison those systems are implicitly making depends on a property of the issue corpus that none of them report.

Two further qualifications matter. Our task hands every structural method a correct seed file for free, whereas LocAgent, CoSIL and the rest must localize from the issue text alone; their task is harder, and a component that underperforms given a free seed might still earn its place without one. And they are LLM agents that also read the issue, so the graph is one component among several. We do not claim their advertised component fails to do its work. The narrower claim we will defend is that on the ground truth those systems themselves adopt, graph proximity is not obviously the strongest available signal even when handed the answer's neighbourhood, and that nobody has reported the comparison or the leakage rate.

Filename leakage deserves to be a standard control. Two tasks in five in this corpus have an issue that names a key file as code. That is not a benchmark defect — real reports really do contain stack traces — but it means any lexical baseline evaluated without the stratification is reporting a mixture of two very different regimes, and any structural method compared against such a baseline is being judged against a moving target. The control costs a regular expression over text the benchmark already ships. We think retrieval papers should report the rate and the stratified result as a matter of course, in the same way they report the corpus size.

What “within two hops” is worth. The two-hop neighbourhood earns its reputation on recall and loses it on precision: it contains seven of every eight needed files and also half the repository. Reporting the recall without the size is, on this corpus, reporting half a statistic.

What this means for an agent's context, and what it does not. The budget framing is not decoration: at a median 286,385 tokens, most of these repositories do not fit in the windows current systems run with, so selection is forced rather than optional. Two numbers bound what selection can achieve. The answer is a median 2.98% of the repository, so a well-chosen context could be some 34× smaller than the whole; and an oracle ordering fits the complete answer inside 8,000 lines for 97.5% of pull requests where our best method manages 58.5%.

We are deliberately not claiming that better selection raises repair rates. We did not run an agent, and a retrieval improvement need not survive contact with one. What the ceiling licenses is the weaker statement: a file absent from the context cannot be used, so the share of tasks whose answer fits the budget bounds any system operating under it. It does not license the claim that the remaining gap is reachable from the signals we have, only that it is not closed off by the tasks themselves. Section 6 specifies the experiment that would convert the bound into a performance claim, in advance of running it.

A note on order versus selection. An ordering plus a budget is a selection, and it is the selection that matters here. We make no claim about the position of a file *within* a prompt, which is a different question with its own literature; every result in this paper concerns which files are read at all.

The human reading of the same result. The budget for a developer is attention rather than tokens, and the results have a plain reading there. Asked to fix the login page, a developer thinks of the authentication files partly because the words match and partly because the imports connect; our fusion result says those two routes are largely complementary, and our leakage result says that how much the words help depends on how explicitly the report was written. We report the token framing because it makes the budget concrete and measurable; we do not think the finding is confined to machines.

Recommendations. (1) Report retrieval results separately for at least one structural and one behavioural notion of relevance, and report the worst case across them. (2) Report, and stratify by, whether the query text names the answer files in code. (3) Match populations before comparing across keys; a key that only exists for multi-file changes does not cover the same tasks as one that does not. (4) Treat a query-independent repository prior, not random order, as the floor for structural methods — on symbol-style keys it captures 84% of a personalised walk. (5) Where a graph-based tool orders context by structure alone, add a lexical signal from the issue text and cost-scale the structural side before fusing; where one already uses both, report them separately as well as combined, since on our corpus the combination beats either part under both keys. (6) Report cost in tokens and report the ceiling alongside the score, so a reader can tell a hard task from a poor ordering. (7) Cluster paired tests by repository; pull requests within a repository are not independent, and ignoring that inflates significance by many orders of magnitude.

6 A PRE-REGISTERED PROTOCOL FOR THE DOWNSTREAM TEST

This paper measures retrieval, not repair. The obvious next question — whether a context assembled by one of these orderings lets a model actually fix the issue — requires running models, which we have not done. Rather than gesture at it as future work, we specify it here in enough detail to be executed or contradicted, in the same spirit as the design freeze that preceded this study (§3). We commit to reporting the result whichever way it falls.

Corpus. The 1,162 scored pull requests of this study. Every one carries a test specification from the benchmark suite it came from, naming the tests that must change from failing to passing and those that must keep passing, so resolution can be decided by execution rather than by patch similarity. We note that only 15 of them overlap SWE-bench Verified, so this is not a re-run of an existing leaderboard.

Conditions. Six contexts per task, each truncated to the same token budget: (i) the seed file alone, a floor; (ii) seed plus the best structural ordering (ppr_und_pl on the symbol key, hops_lines on the co-edited key — no single structural ordering is best under both, so the condition is defined per key); (iii) seed plus the best lexical ordering (path_issue); (iv) seed plus the best fusion by worst-case rank (rrf_hops_path); (v) seed plus the true key, a ceiling; (vi) the repository truncated to the budget in a fixed canonical order, which is what a system with no retrieval step does. Budgets of 16,000 and 64,000 tokens, chosen from Table 9 to sit either side of the point where most answers begin to fit.

Models. Two, of clearly different capability, so that the result is not a property of one model. A weak model that fails everywhere and a strong model that succeeds everywhere would both be uninformative; the pilot below exists to detect that before the full run.

Generation. A single API call per task and condition, with no tools and no retrieval available to the model. This is essential rather than incidental: a model that can search the repository defeats the budget, and the quantity under study is what a fixed context supports.

Outcome and analysis. A task is resolved if the generated patch applies and the specified tests pass. The comparison is paired across conditions on the same tasks, tested with McNemar’s test, Holm-corrected across the condition pairs, with the pull request as the unit as elsewhere in this paper. We do not require the models to be reliable; we require the *difference between conditions* to be measurable, and the pairing makes the stochastic component common to all conditions.

Pilot and stopping rule. A 150-task pilot on the weaker model precedes the full run. If the oracle condition resolves under 10% or over 90% of pilot tasks, the budget is outside the informative range and is adjusted before the main run — and that adjustment is reported.

What would falsify the implication. If conditions (ii), (iii) and (iv) do not differ from (vi) at equal budget, then retrieval quality does not transfer to repair at these budgets, and the results of this paper should be read as being about retrieval alone. That outcome is publishable and we would report it.

7 THREATS TO VALIDITY

Construct. Neither evidence source is ground truth about what a developer needed to read; they are recoverable proxies with opposite biases, which is why we report both and never merge them. Coverage per line assumes whole files are read, which overstates the cost for agents that read selectively. The execution-based source is absent (§3.3) and would be the natural third leg.

Internal. The graph builder resolves imports statically and misses dynamic imports and plugin registration. The symbol key is produced by a static analyser and inherits its resolution failures. Both biases favour structural methods. Issue-path leakage is measured and does substantially drive the co-edited result: it is reported in Section 4.9 as a finding rather than a reassurance, and the corresponding claim is stated conditionally throughout. Our leakage classifier is a regular expression over issue text and will mislabel some mentions in either direction; we report three nested strata rather than one so that the conclusion does not rest on a single cut.

External. Python only, file granularity only, and pull requests drawn from benchmark suites that select for reproducible test-verified fixes — a population plausibly better documented than average, which may flatter the issue-text methods. The median repository here has 131 graph files and very large repositories are under-represented. The size split in Section 4.10 shows the pattern to be stable across size quartiles rather than widening with size, so we cannot argue that the bias runs in a harmless direction; we can only say that it does not appear to be a size effect.

Conclusion. Pull-request-level aggregation, repository-level paired tests, Holm correction within comparison families and bootstrapping over repositories guard against the main inflation risks. Effect sizes accompany every test because several differences are significant and small. The fusion family, the worst-rank criterion and the directed walks were all added after the design freeze and are exploratory; we have not held out a split of repositories to confirm them, which is the main remaining weakness of the fusion claim.

8 CONCLUSION

We compared import-graph proximity and issue-text retrieval as reading orders for Python change tasks on 2,412 tasks from 1,162 pull requests across 112 repositories, under two evidence-based definitions of which files a change required. On the full corpus the two definitions give opposite

answers, structural orderings winning on the files a change references and lexical orderings on the files a developer changed alongside it. Most of this paper is then about why that headline does not survive contact with two controls. Matching the populations, since a co-edited key only exists for a multi-file change, removes the structural advantage on the symbol key. Removing the two tasks in five whose issue names a key file in code removes the lexical advantage on the co-edited key. Each control dissolves one half, and neither family dominates once both are applied.

What survives is the combination. Reciprocal-rank fusion of a cost-scaled walk with issue-text-to-path matching is the best non-oracle ordering under both keys on a matched population, holds the top places under every leakage stratum, and costs a BM25 index over the file list. The graph's two-hop neighbourhood, the standard justification for structural retrieval, turns out to be half the repository, and an oracle fits the complete answer in 8,000 lines for 97.5% of pull requests where our best method reaches 58.5%.

We recommend that context-retrieval work for code change report per evidence source, report and stratify by whether the query names the answer, match populations before comparing across keys, cluster its tests by repository, use a repository prior rather than random order as its floor, and stop treating the dependency graph as the only signal in the room.

DATA AVAILABILITY

The replication package contains the import graphs at every base commit, the task definitions and evidence keys, the issue-token bags, the per-task orderings and coverage curves, the per-file token counts, and the analysis scripts that produce every number and figure in this paper. It will be archived at a permanent identifier on acceptance.

USE OF AI TOOLS

Claude (Anthropic) was used to write the analysis and method code, to run the experiments and to draft this manuscript. All numeric results are produced by scripts in the replication package and were copied mechanically from their output files; the author verified the design, the claims and the text. Bibliographic entries were drafted with AI assistance and then checked against arXiv, the ACL Anthology and the publisher listings on 2026-09-23; the evaluation section of every system in Table 1 was read directly rather than summarised from an abstract. That check corrected eight entries drafted from memory, including a journal name, an author order, three titles changed between preprint versions, and one volume recorded as authored that is in fact edited. The corrections are listed in the replication package.

REFERENCES

- [1] Aider. 2023. Building a Better Repository Map with Tree Sitter. <https://aider.chat/2023/10/22/repomap.html>. Blog post, 22 October 2023. Accessed 2026-09-23.
- [2] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2025. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. arXiv:2505.20411 [cs.SE]
- [3] Shawn A. Bohner and Robert S. Arnold (Eds.). 1996. *Software Change Impact Analysis*. IEEE Computer Society Press, Los Alamitos, CA.
- [4] Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, Jie Wang, Xiao Cheng, Guangtai Liang, Yuchi Ma, Pan Bian, Tao Xie, and Qianxiang Wang. 2024. CodeR: Issue Resolving with Multi-Agent and Task Graphs. arXiv:2406.01304 [cs.SE]
- [5] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. 2025. LocAgent: Graph-Guided LLM Agents for Code Localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 8697–8727. <https://doi.org/10.18653/v1/2025.acl-long.426> Introduces the Loc-Bench benchmark. arXiv:2503.09089.

- [6] Bogdan Dit, Meghan Revelle, Malcom Gethers, and Denys Poshyvanyk. 2013. Feature Location in Source Code: A Taxonomy and Survey. *Journal of Software: Evolution and Process* 25, 1 (2013), 53–95. <https://doi.org/10.1002/smr.567>
- [7] Kang He and Kaushik Roy. 2026. SWE-Adept: An LLM-Based Agentic Framework for Deep Codebase Analysis and Structured Issue Resolution. arXiv:2603.01327 [cs.SE]
- [8] Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. 2025. Issue Localization via LLM-Driven Iterative Code Graph Searching. arXiv:2503.22424 [cs.SE] Earlier version titled “CoSIL: Software Issue Localization via LLM-Driven Code Repository Graph Searching”. To appear, ASE 2025.
- [9] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world GitHub Issues?. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.06770. SWE-bench Lite is released with this benchmark.
- [10] Amy J. Ko, Brad A. Myers, Michael J. Coblenz, and Htet Htet Aung. 2006. An Exploratory Study of How Developers Seek, Relate, and Collect Relevant Information during Software Maintenance Tasks. *IEEE Transactions on Software Engineering* 32, 12 (Dec. 2006), 971–987. <https://doi.org/10.1109/TSE.2006.116>
- [11] Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. 2026. ContextBench: A Benchmark for Context Retrieval in Coding Agents. arXiv:2602.05892 [cs.SE]
- [12] Wei Liu, Chao Peng, Pengfei Gao, Aofan Liu, Wei Zhang, Haiyan Zhao, and Zhi Jin. 2025. GraphLocator: Graph-guided Causal Reasoning for Issue Localization. arXiv:2512.22469 [cs.SE]
- [13] Yingwei Ma, Qingping Yang, Rongyu Cao, Binhua Li, Fei Huang, and Yongbin Li. 2025. Alibaba LingmaAgent: Improving Automated Issue Resolution via Comprehensive Repository Exploration. In *Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25), Industry Track*. <https://doi.org/10.1145/3696630.3728549> arXiv:2406.01422 (v1 titled “How to Understand Whole Software Repository?”).
- [14] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. 2025. RepoGraph: Enhancing AI Software Engineering with Repository-Level Code Graph. In *International Conference on Learning Representations (ICLR)*. arXiv:2410.14684.
- [15] Bowen Qin and Yi Xie. 2026. Agent Retrieval Bench: Evaluating Repository Context Retrieval for Coding Agents. arXiv:2607.24882 [cs.SE]
- [16] Revanth Gangi Reddy, Tarun Suresh, JaeHyeok Doo, Ye Liu, Xuan Phi Nguyen, Yingbo Zhou, Semih Yavuz, Caiming Xiong, Heng Ji, and Shafiq Joty. 2025. SweRank: Software Issue Localization with Code Ranking. arXiv:2505.07849 [cs.SE]
- [17] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389. <https://doi.org/10.1561/15000000019>
- [18] Martin P. Robillard. 2008. Topology Analysis of Software Dependencies. *ACM Transactions on Software Engineering and Methodology* 17, 4 (Aug. 2008), 18:1–18:36. <https://doi.org/10.1145/13487689.13487691>
- [19] Lintang Sutawika, Aditya Bharat Soni, Bharath Sriraam R R, Apurva Gandhi, Taha Yassine, Sanidhya Vijayvargiya, Yuchen Li, Xuhui Zhou, Yilin Zhang, Leander Melroy Maben, and Graham Neubig. 2026. CodeScout: An Effective Recipe for Reinforcement Learning of Code Search Agents. arXiv:2603.17829 [cs.SE]
- [20] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE (July 2025), FSE037. <https://doi.org/10.1145/3715754> Agentless. arXiv:2407.01489.
- [21] Chengxing Xie, Bowen Li, Chang Gao, He Du, Wai Lam, Difan Zou, and Kai Chen. 2025. SWE-Fixer: Training Open-Source LLMs for Effective and Efficient GitHub Issue Resolution. In *Findings of the Association for Computational Linguistics: ACL 2025*. 1123–1139. <https://doi.org/10.18653/v1/2025.findings-acl.62> arXiv:2501.05040.
- [22] Boyang Yang, Jiadong Ren, Shunfu Jin, Yang Liu, Feng Liu, Bach Le, and Haoye Tian. 2025. Enhancing Repository-Level Software Repair via Repository-Aware Knowledge Graphs. arXiv:2503.21710 [cs.SE] KGCompass. An earlier version was titled “KGCompass: Knowledge Graph Enhanced Repository-Level Software Repair”.
- [23] Annie T. T. Ying, Gail C. Murphy, Raymond Ng, and Mark C. Chu-Carroll. 2004. Predicting Source Code Changes by Mining Change History. *IEEE Transactions on Software Engineering* 30, 9 (Sept. 2004), 574–586. <https://doi.org/10.1109/TSE.2004.52>
- [24] Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. 2025. SWE-bench Goes Live! arXiv:2505.23419 [cs.SE] NeurIPS 2025 Datasets and Benchmarks Track (per the authors’ repository).
- [25] Shaoqiu Zhang, Yuhang Wang, Jialiang Liang, Yuling Shi, Wenhao Zeng, Maoquan Wang, Shilin He, Ningyuan Xu, Siyu Ye, Kai Cai, and Xiaodong Gu. 2026. SWE-Explore: Benchmarking How Coding Agents Explore Repositories. arXiv:2606.07297 [cs.SE]

- [26] Thomas Zimmermann, Peter Weißgerber, Stephan Diehl, and Andreas Zeller. 2004. Mining Version Histories to Guide Software Changes. In *Proceedings of the 26th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Edinburgh, United Kingdom, 563–572.